


```
fun calculadora(
    x:Int,
    y:Int,
    operacion: (Int, Int) -> Int):Int{
    return operacion(x,y)
}

fun sumar(a: Int, b: Int): Int {
    return a + b
}

fun restar(a: Int, b: Int): Int {
    return a - b
}

fun main(){
    print(calculadora(4,5, ::restar))
}
```

Funciones de Orden Superior y Lambdas

3

INTRODUCCION

Aunque Kotlin es un lenguaje orientado a objetos, introduce características existentes en los lenguajes funcionales que nos permiten crear un código más claro y expresivo, lo cual es fundamental para el desarrollo moderno en Android.

Una de las características del paradigma de la programación funcional son las funciones de orden superior.

Las funciones de orden superior y las expresiones lambda son el núcleo de la programación funcional en Kotlin y representan una de las mayores diferencias y ventajas sobre Java tradicional.

FUNCIONES DE ORDEN SUPERIOR

Una función de orden superior es una función que puede tomar otras funciones como argumentos o devolver funciones como resultado. Este enfoque permite la composición flexible de comportamientos, lo que facilita la creación de APIs muy claras y el manejo eficiente de colecciones de datos.

Por ejemplo, en Android, cuando se trabaja con listas grandes, se usan funciones de orden superior como map, filter o forEach para transformar o seleccionar elementos sin tener que escribir bucles manualmente.

FUNCIONES DE ORDEN SUPERIOR

Imaginemos una función que calcula el **doble** de un número. Además, tenemos la función **aplicarOperacion** es una función de orden superior que toma como parámetros un **número** entero y una función que recibe un entero y devuelve otro entero **operación: (Int) -> Int**.

```
fun doble(numero: Int): Int {
     return numero * 2
}
```

```
fun aplicarOperacion(numero: Int, operacion: (Int) -> Int): Int {
    return operacion(numero)
}
```

```
fun main() {
   val valorInicial = 5

   val resultadoDoble = aplicarOperacion(valorInicial, ::doble)
   println("Aplicando 'doble' a $valorInicial: $resultadoDoble")
}

// Salida
// Aplicando 'doble' a 5: 10
```

Se usa el operador '::' para la referencia

EXPRESIONESIONES LAMBDA

Una expresión lambda (o lambda) es una función anónima, esencialmente un bloque de código que se puede tratar como un valor: se puede pasar como argumento, devolverlo desde una función o almacenarlo en una variable.

En Kotlin, las lambdas se utilizan para implementar los parámetros de función de las funciones de orden superior, haciendo que el código sea extremadamente conciso. Son la forma en que se escriben los listeners de eventos en Android de manera moderna y limpia, reemplazando las clases anónimas de Java.

Una lambda se define utilizando la notación:

{ argumentos -> cuerpo de la función }.

EXPRESIONESIONES LAMBDA

Vamos a hacer una expresión Lambda que calcule el doble de un número.

```
val doble = {numero: Int -> numero * 2}
```

Recordemos nuestra función de orden superior aplicarOperacion:

```
fun aplicarOperacion(numero: Int, operacion: (Int) -> Int): Int {
    return operacion(numero)
}
```

Podemos pasar la **expresión Lambda** como argumento de la función:

```
val resultado = aplicarOperacion(5){numero -> numero * 2}
//resultado toma el valor de 10
```

EXPRESIONESIONES LAMBDA

Escribimos el ejemplo del cálculo del doble que vimos antes con funciones Lambda, y añadimo sotra que calcula el triple de un número entero:

```
fun aplicarOperacion(numero: Int, operacion: (Int) -> Int): Int {
    return operacion(numero)
}
```

```
fun main() {
   val valorInicial = 5

   val resultadoLambdaDoble = aplicarOperacion(valorInicial) { n -> n * 2 }
   println("Aplicando lambda doble a $valorInicial: $resultadoLambdaDoble")

   val resultadoLambdaTriple = aplicarOperacion(resultadoLambdaDoble) { n -> n * 3 }
   println("Aplicando lambda triple a 10: $resultadoLambdaTriple")
}
// Salida
// Aplicando lambda doble a 5: 10
// Aplicando lambda triple a 10: 30
```

EXPRESIONESIONES LAMBDA: Kotlin y Java

Veamos el mismo código de antes en Kotlin y en Java:

```kotlin
fun aplicarOperacion(numero: Int, operacion: (Int) -> Int): Int {
    return operacion(numero)
}
fun main() {
   val valor = 5

   val rdoDoble = aplicarOperacion(valor) { n -> n * 2 }
   println("Aplicando lambda doble a $valor: $rdoDoble")

   val rdoTriple = aplicarOperacion(rdoDoble) { n -> n * 3 }
   println("Aplicando lambda triple a 10: $rdoTriple")
}
```

```java
import java.util.function.IntUnaryOperator;

public class Main {

    public static int aplicarOperacion(int valor, IntUnaryOperator operacion) {
        return operacion.applyAsInt(valor);
    }

    public static void main(String[] args) {
        int valor = 5;

        int rdoDoble = aplicarOperacion(valor, n -> n * 2);
        System.out.println("Aplicando lambda doble a " + valor + ": " + rdoDoble);

        int rdoTriple = aplicarOperacion(rdoDoble, n -> n * 3);
        System.out.println("Aplicando lambda triple a 10: " + rdoTriple);
    }
}
```

EJEMPLO

```
fun operaciones (v1: Int, v2: Int, fn: (Int, Int) -> Int) : Int {
    return fn(v1, v2)
}

fun sumar(x1: Int, x2: Int) = x1 + x2

fun restar(x1: Int, x2: Int) = x1 - x2

fun multiplicar(x1: Int, x2: Int) = x1 * x2

fun dividir(x1: Int, x2: Int): Int {
    var rdo = 0
    if (x2 != 0) rdo = x1/x2 else println("No se puede dividir por 0. El resultado será 0")
    return rdo
}

fun main(parametro: Array<String>) {
     val resSuma = operaciones(10, 5, ::sumar)
     println("La suma de 10 y 5 es $resSuma ")
     val resResta = operaciones(5, 2, ::restar)
     println("La resta de 5 y 2 es $resResta ")
     println("El producto de 5 y 20 es ${operaciones(5, 20, ::multiplicar)}")
     println("La división de 10 entre 5 es ${operaciones(10, 5, ::dividir)}")
}
```

EJEMPLO

```
fun operaciones(v1: Int, v2: Int, fn: (Int, Int) -> Int) : Int {
    return fn(v1, v2)
}

fun main() {
    val num1 = 10
    val num2 = 5

    val resSuma = operaciones(10, 5) { a, b -> a + b }
    println("La suma de 10 y 5 es $resSuma")

    val resResta = operaciones(5, 2) { a, b -> a - b }
    println("La resta de 5 y 2 es $resResta")

    println("El producto de 5 y 20 es ${operaciones(5, 20) { a, b -> a * b }}")
    val resDivision = operaciones(10, 0) { dividendo, divisor ->
        if (divisor != 0) {
            dividendo / divisor
        } else {
            println("No se puede dividir por 0. El resultado será 0")
            0 // Devuelve 0 en caso de división por cero
        }
    }
    println("La división de 10 entre 0 es $resDivision")
}
```

map, filter y forEach

filter, map y forEach son funciones de orden superior de Kotlin para trabajar con colecciones. Nos permiten manipular datos de forma declarativa, es decir, dices qué quieres hacer en lugar de cómo hacerlo (como harías con un bucle for tradicional).

map y filter no modifican la lista original, sino que crean una nueva lista como resultado:

- filter (Filtrar): Devuelve una nueva lista que contiene solo los elementos de la lista original que cumplieron una condición (predicado). El número de elementos puede ser igual o menor.
- map (Transformar): Devuelve una nueva lista que tiene exactamente el mismo número de elementos que la original, pero cada elemento ha sido transformado (mapeado) a un nuevo valor.

forEach es diferente. No devuelve una lista nueva. De hecho, no devuelve nada útil (devuelve Unit, que es el equivalente a void). Su utilidad es ejecutar una por cada elemento de una lista (imprimirlo, guardarlo en una base de datos, añadirlo a otra lista mutable, etc.)..

map

map recorre cada elemento de la lista y le aplica una transformación (la operación que se le pasa como expresión).

El resultado es siempre una nueva lista con el mismo número de elementos que la original, pero cada elemento ha sido transformado en algo nuevo.

```
fun main() {
    val nombres = listOf("Laura", "Ana", "Silvia", "Nerea")
    val numeros = nombres.map {it.length}

    val numerosDuplicados = numeros.map { it * 2 }

    println("Lista original:                $nombres")
    println("Lista longitudes:              $numeros")
    println("Lista transformada:   $numerosDuplicados")
}
```

filter

filter recorre una lista y mira cada elemento, revisando si cumple la condición que le pasamos:

- Si el elemento cumple la condición (devuelve true), y se incluye en la nueva lista.
- Si no la cumple (devuelve false), se queda fuera.

El resultado es siempre una nueva lista que contiene solo los elementos cumplen la condición. Esta lista puede tener menos elementos o los mismos, pero los elementos en sí no cambian.

```
fun main() {
    val numeros = listOf(1, 2, 3, 4, 5, 6, 7, 8)

    val numerosPares = numeros.filter { it % 2 == 0 }

    println(“Números originales: $numeros")
    println(“Números filtrados:  $numerosPares")
}
```

forEach

forEach recorre cada elemento de la lista y simplemente hace algo con él (como imprimirlo, guardarlo en otro sitio, etc.).

A diferencia de map y filter, forEach no devuelve una lista nueva.

```
class Persona(val nombre: String, val edad: Int, var mayor: Boolean){
    override fun toString(): String{
        return "[$nombre, $edad, $mayor]"
    }
}
fun main(){
    val personas = listOf(
        Persona("Juan", 25, false),
        Persona("Ana", 17, false),
        Persona("Luis", 30, false),
        Persona("María", 22, false)
    )
    println("Lista original")
    personas.forEach {println(it)}
    personas.forEach { if (it.edad > 18) it.mayor = true else it.mayor = false }
    println("Lista actualizada:")
    personas.forEach {println(it)}
}
```

EJEMPLOS PRÁCTICOS

Mapeo de una Lista

Supongamos que tenemos una lista de números y queremos aplicar una operación a cada elemento de la lista. Podemos utilizar la función **map** de Kotlin junto con una lambda para lograr esto de manera elegante:

```
fun main(){
    val numeros = listOf(1, 2, 3, 4, 5)
    println("Lista base: $numeros")
    val duplicados = numeros.map { numero -> numero * 2 }
    println("Lista transformada: $duplicados")
}
```

salida

```
Lista base: [1, 2, 3, 4, 5]
Lista transformada: [2, 4, 6, 8, 10]
```

En este caso, la función **map** toma una lambda que especifica cómo se debe transformar cada elemento de la lista.

El resultado es una nueva lista con los elementos transformados.

EJEMPLOS PRÁCTICOS

Filtrado de una Lista

Supongamos que tenemos una lista de personas y queremos filtrar aquellas que son mayores de 18 años:

```
class Persona(val nombre: String, val edad: Int){
    override fun toString(): String{
        return "[$nombre, $edad]"
    }
}
fun main(){
    val personas = listOf(
        Persona("Juan", 25),
        Persona("Ana", 17),
        Persona("Luis", 30),
        Persona("María", 22)
    )
 println("Todas:")
    println(personas)
    val mayoresDeEdad = personas.filter { persona -> persona.edad >= 18 }
    println("Mayores:")
    println(mayoresDeEdad)
}
```

salida

```
Todas:
[[Juan, 25], [Ana, 17], [Luis, 30], [María, 22]]
Mayores:
[[Juan, 25], [Luis, 30], [María, 22]]
```

En este caso, utilizamos la función **filter** junto con una lambda que especifica el criterio de filtrado.

El resultado es una nueva lista que contiene solo las personas que cumplen con el criterio.

EJEMPLOS PRÁCTICOS

Filtrado de una Lista

Supongamos que tenemos una lista de personas y queremos filtrar aquellas que son mayores de 18 años:

```
class Persona(val nombre: String, val edad: Int){
    override fun toString(): String{
        return "[$nombre, $edad]"
    }
}
fun main(){
    val personas = listOf(
        Persona("Juan", 25),
        Persona("Ana", 17),
        Persona("Luis", 30),
        Persona("María", 22)
    )
 println("Todas:")
    println(personas)
    val mayoresDeEdad = personas.filter { persona -> persona.edad >= 18 }
    println("Mayores:")
    println(mayoresDeEdad)
}
```

salida

```
Todas:
[[Juan, 25], [Ana, 17], [Luis, 30], [María, 22]]
Mayores:
[[Juan, 25], [Luis, 30], [María, 22]]
```

En este caso, utilizamos la función **filter** junto con una lambda que especifica el criterio de filtrado.

El resultado es una nueva lista que contiene solo las personas que cumplen con el criterio.

EJEMPLOS PRÁCTICOS

Manejo de Eventos en Android

En el desarrollo de aplicaciones Android, las funciones de orden superior y las lambdas son especialmente útiles para manejar eventos.

Por ejemplo, al manejar el clic en un botón, se pueden utilizar lambdas para definir el comportamiento que debe ocurrir cuando se pulsa ese botón:

```
boton.setOnClickListener { view ->
   // Código a ejecutar cuando se pulsa el botón
}
```

Como no estamos en Android, vamos a simular el comportamiento con Kotlin:

```
fun setOnClickListenerZ(contador: Int, fn: (Int) -> Int): Int {
   return fn(contador)
}
```

```
fun main() {
    var contador = 0
    println("Contador inicial: $contador")

    contador = setOnClickListenerZ(contador) { c ->
        println("Incrementando el valor en 1.")
        c+1
    }
    println("Contador +1: $contador")

    contador = setOnClickListenerZ(contador) { c ->
        println("Incrementando el valor en 5.")
        c+5
    }
    println("Contador +5: $contador")
}
```

salida

```
Contador inicial: 0
Incrementando el valor en 1.
Contador +1: 1
Incrementando el valor en 5.
Contador +5: 6
```

Ejercicios

1. Crea un programa en Kotlin que gestione una lista de precios y aplique un descuento del 10% únicamente a los productos cuyo precio inicial supere los 50.
2. Crea un programa en Kotlin que gestione una lista de nombres de usuario. El objetivo es procesar cada nombre para clasificar si es "corto" o "largo" y escribirlo en mayúscula.

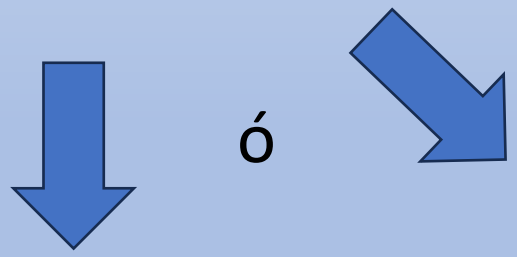
Si el nombre tiene 5 caracteres o menos, se clasifica como "Corto". Si

el nombre tiene más de 5 caracteres, se clasifica como "Largo".

Devuelve cada elemento como un String formateado, que incluya la

clasificación y el nombre original en MAYÚSCULAS.

```
val nombres = listOf("Nom1", "Nomb2", "Nombr3", "Nombre4")
```



ó


```
Resultados de la clasificación:
[Corto] Nombre: NOM1
[Corto] Nombre: NOMB2
[Largo] Nombre: NOMBR3
[Largo] Nombre: NOMBRE4
```

```
Resultados de la clasificación: [[Corto] Nombre: NOM1, [Corto] Nombre: NOMB2, [Largo] Nombre: NOMBR3, [Largo] Nombre: NOMBRE4]
```